

A Mini Project Report

On

Verdict Vault

Submitted by

Sampath G L (225891407)

Under the guidance of

Dr. Anitha Premkumar
Assistant Professor – Senior Scale
School of Computer Engineering

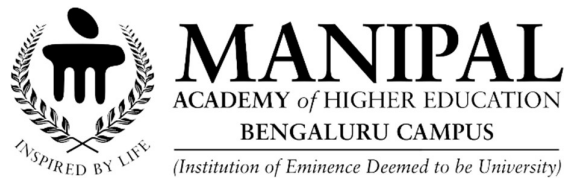
In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (CYBERSECURITY)

At



Department of Information Technology
Manipal Institute of Technology, Bengaluru
Oct 2025

CERTIFICATE

This is to certify that the project entitled “**Verdict vault**” is a bonafide work carried out as part of the course **Blockchain Technology(Ethereum Smart Contract Using Solidity-IT_4421)**, under my guidance by **Sampath G L (225891407)**, student of VII Sem B.Tech (CSE-CYB) at the Department of Information Technology, Manipal Institute of Technology, Bengaluru during the academic semester 2025, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Specializing in Cybersecurity), at MIT Bengaluru.

Place: Bangalore

Date:13-10-2025

Signature of the Instructor

DECLARATION

We hereby declare that the project entitled “Verdict Vault” submitted as a part of the partial course requirements for the course **Blockchain Technology(Ethereum Smart Contract Using Solidity-IT_4421)** for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Specializing in Cybersecurity) at MIT Bengaluru during the **Jul – Dec 2025** semester has been carried out by us. We declare that the project has not formed the basis for the award of any degree, scholarship, fellowship, or any other similar titles elsewhere.

Further, we declare that we will not share, re-submit, or publish the code, idea, framework, and/or any publication that may arise out of this work for academic or profit purposes without obtaining the prior written consent of the Course Faculty Mentor and Course Instructor.

Signature of the Students:

Sampath G L

Place: Bangalore

Date: 13-10-2025

TABLE OF CONTENTS

SL. NO	CONTENTS	PAGE NO.
1	Introduction 1.1 Scope of the Work 1.2 Usage Scenarios	5 5 5
2	Requirement Analysis 2.1 Functional Requirements 2.2 Non-functional Requirements	6 6 6
3	Design and Development 3.1 System architecture <i>FlowChart1: System Architecture</i> 3.2 Key Modules <i>Table1: Key Modules Table</i> 3.3 Technologies Used 3.4 On-Chain vs Off-Chain Storage in VerdictVault <i>FlowChart2: Work Flow Diagram</i>	7 7 7 8 8 8 9 10
4	Work Demo 4.1 Uploading and Decrypting <i>Fig1: Document Uploading Process</i> <i>FlowChart3: Encryption Flow</i> 4.2 Decrypting and Downloading <i>Fig2: Document Decrypting Process</i> <i>Fig3: Transactions Proof</i> <i>FlowChart4: Decryption Flow</i>	11 11 11 11 12 12 13 13
5	Conclusion and Future Work 6.1 Conclusion 6.2 Future Work	14 14 14
6	References	15
7	Appendix 7.1 Features Table <i>Table2: Features Table</i>	16 16 16

1 INTRODUCTION

Verdict Vault is a digital evidence management system designed to streamline the storage, retrieval, and tracking of digital forensic evidence in compliance with legal standards. It offers a secure, scalable solution for law enforcement agencies, legal professionals, and organizations dealing with cybercrime investigations. The system is built to ensure tamper-proof evidence storage while providing an easy-to-use interface for managing digital evidence through secure access and traceable audit trails.

1.1 Scope of the Work

The goal of this project is to design and implement a secure and scalable digital evidence management system that adheres to industry standards for security, compliance, and usability. The system allows for the secure submission, tracking, and retrieval of digital evidence, ensuring that all actions taken on the evidence are logged and auditable.

Key features include:

- Secure storage of digital evidence (files, logs, etc.) with encryption.
- Auditable workflows for handling evidence, ensuring that every action is logged.
- User roles and access control, with detailed permissions for system users.
- Search and retrieval capabilities that allow easy access to specific pieces of evidence based on metadata and other attributes.
- Integration with existing law enforcement or legal systems for streamlined case management.

Verdict Vault is designed with compliance to relevant legal standards, including data protection laws, encryption standards, and evidence handling protocols.

1.2 Usage Scenarios

Verdict Vault can be employed in a range of digital forensic scenarios, including but not limited to:

Law Enforcement Investigations

Law enforcement agencies can use Verdict Vault to manage digital evidence from cybercrime investigations. The system ensures evidence is securely stored, traceable, and available for review during the investigation or trial.

Legal Professionals and Courts

Lawyers and court officials can utilize Verdict Vault to handle evidence presented during trials. The system allows for easy retrieval and presentation of digital evidence while ensuring integrity and compliance.

Corporate Investigations

In corporate settings, internal investigations into data breaches, fraud, or employee misconduct can benefit from Verdict Vault's secure evidence management capabilities, ensuring confidentiality and data integrity.

Educational and Research Institutions

Educational institutions teaching digital forensics can use Verdict Vault as a teaching tool, providing students with real-world scenarios for managing digital evidence while maintaining a high level of security.

2 REQUIREMENT ANALYSIS

2.1 Functional Requirements

These are the core functionalities that Verdict Vault must support:

- **Evidence Submission:** Users must be able to securely upload evidence files to the system, with automatic encryption.
- **Evidence Tracking:** Each piece of evidence must be assigned a unique identifier and tracked throughout its lifecycle.
- **Role-Based Access Control (RBAC):** The system must provide role-based user access to ensure that only authorized personnel can access or modify specific evidence.
- **Audit Logging:** Every action taken on evidence must be logged, including who performed the action and the time it was done.
- **Search Functionality:** The system must allow users to search for evidence based on metadata, such as case number, file type, or date of submission.
- **Evidence Retrieval:** Users with appropriate access rights should be able to retrieve evidence securely and maintain a clear chain of custody.
- **Reporting:** The system must generate reports on evidence status, actions taken, and audit trails.

2.2 Non-functional Requirements

Scalability: The system must be able to handle large volumes of evidence data without performance degradation.

Security: All data, especially sensitive evidence, must be encrypted both at rest and in transit.

Compliance: The system must comply with industry standards and legal regulations regarding evidence handling, such as GDPR, HIPAA, and local data protection laws.

Usability: The user interface must be intuitive for both technical and non-technical users.

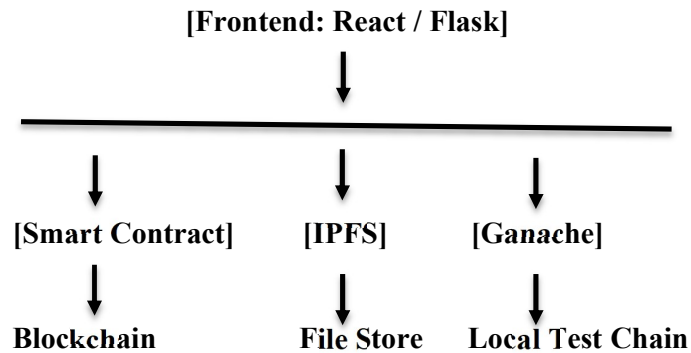
Reliability: The system should be highly available with minimal downtime, particularly when evidence is being retrieved.

Performance: The system should allow for quick evidence retrieval, even when handling large amounts of data.

Auditability: Every action on the system must be logged and retrievable for forensic audits.

3 DESIGN AND DEVELOPMENT

3.1 System Architecture



FlowChart1: System Architecture

Verdict Vault is built on a three-tier architecture:

1. Frontend (User Interface):

- Built using React.js for the web-based interface.
- Provides a clean, easy-to-use interface for evidence submission, retrieval, and audit logs.

2. Backend (Business Logic):

- Developed using Python and Flask for server-side logic.
- Manages evidence storage, user roles, and audit trails.

3. Database Layer:

- Uses a Relational Database Management System (RDBMS) like PostgreSQL to store evidence metadata and audit logs.
- Evidence itself is stored in encrypted form using a distributed storage system such as Amazon S3 or local servers.

3.2 Key Modules

- Evidence Upload: Responsible for securely uploading and encrypting evidence files.
- Evidence Management: Manages metadata and tracking of evidence through its lifecycle.
- User Management: Manages user roles, permissions, and authentication.
- Audit and Compliance: Ensures that every user action is logged and that audit trails are maintained.
- Search and Retrieval: Allows users to search for specific evidence based on predefined filters.
- Reporting: Generates evidence and audit reports for legal or investigative use.

Module	Description	Technologies Used
Evidence Upload	Encrypts and uploads digital files	AES, IPFS
Evidence Management	Handles metadata, lifecycle tracking	PostgreSQL, Flask
User Management	Role-based access and authentication	JWT, Flask-Login
Audit Logging	Records all user and system actions	PostgreSQL Logs
Search & Retrieval	Enables metadata-based querying and download	React.js, Flask API

Table1: Key Modules Table

3.3 Technologies Used

- Frontend: React.js, Material-UI
- Backend: Python, Flask
- Database: PostgreSQL, Amazon S3 (for file storage)
- Authentication: JWT (JSON Web Tokens) for secure user authentication
- Encryption: AES (Advanced Encryption Standard)
- Containerization: Docker for deployment

3.4 On-Chain vs Off-Chain Storage in VerdictVault

In blockchain applications like VerdictVault, data can be stored in two main ways: **on-chain** and **off-chain**. Understanding the difference is essential for optimizing performance, cost, and security.

On-Chain Storage (Smart Contract)

What is stored:

- Case ID
- SHA-256 hash of the encrypted document
- IPFS CID (Content Identifier)
- Metadata (Title, Category)

Characteristics:

- Data is stored directly on the blockchain (e.g., Ganache/Ethereum).
- Immutable and verifiable.
- Ideal for small, critical data (e.g., fingerprints of files).
- Costly if used for storing large files due to gas fees.
- Used to validate file authenticity and traceability.

Off-Chain Storage (IPFS)

What is stored:

- The actual **encrypted document**

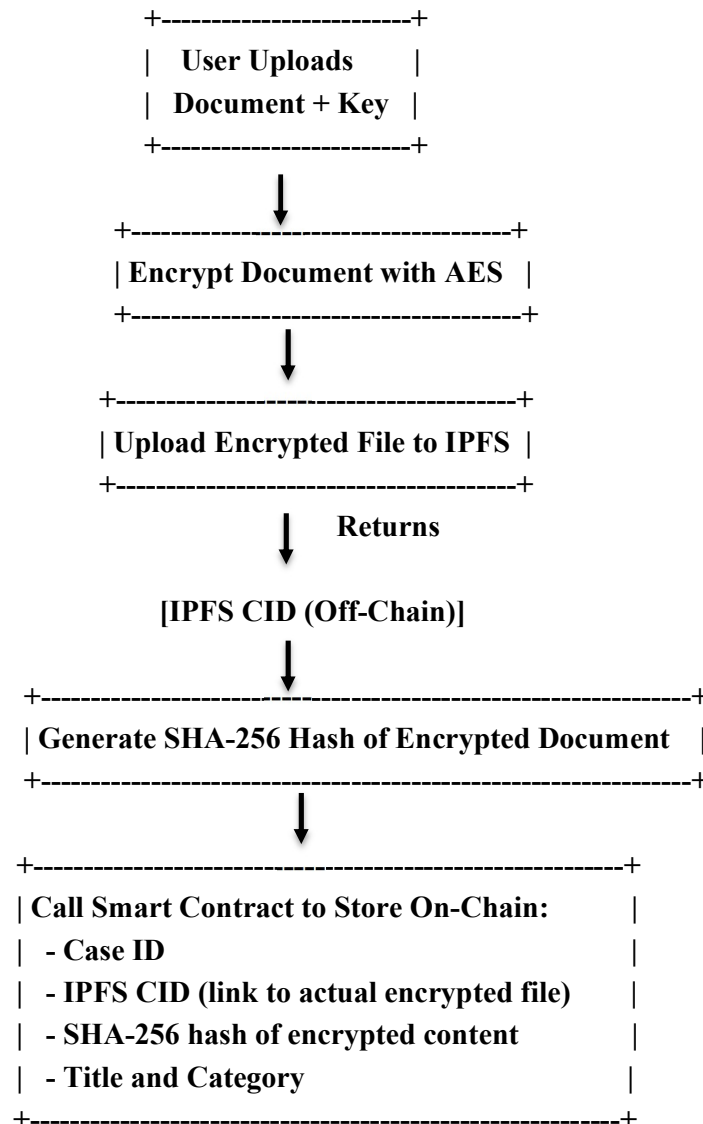
Characteristics:

- Stored in the InterPlanetary File System (IPFS), a decentralized file storage system.
- Returns a unique CID for every uploaded file based on its content.
- Efficient for storing large files like PDFs, images, or multimedia.
- Reduces blockchain storage cost significantly.
- Accessed via the CID stored on-chain.

Why This Hybrid Model Is Used in VerdictVault ?

- The **encrypted document** is too large to store on-chain, so it is stored **off-chain** using IPFS.
- The **SHA-256 hash** (file fingerprint), **IPFS CID**, and **metadata** are stored **on-chain** for tamper-proof verification.
- This enables **blockchain-level security and transparency**, while remaining **cost-efficient and scalable**.

Visual Work Flow On-Chain vs Off-Chain Storage in VerdictVault



FlowChart2: Work Flow Diagram

4 WORK DEMO

Here's what happens behind the scenes when someone uploads a document:

4.1 Uploading and Encrypting Document

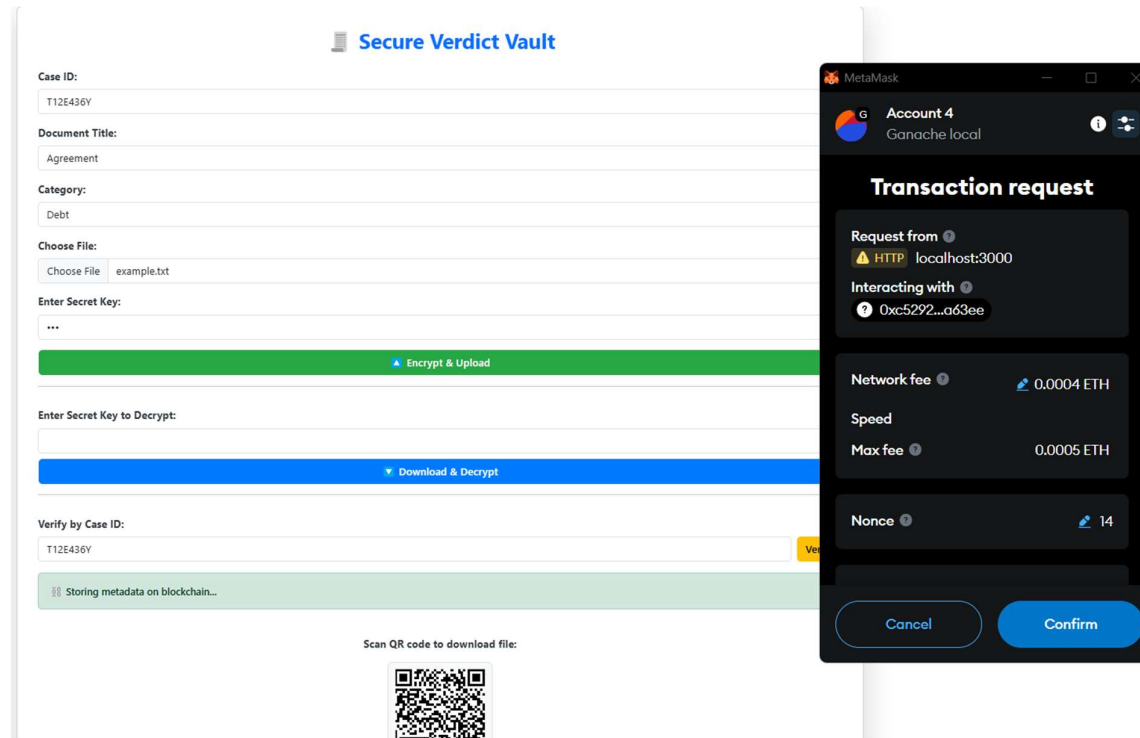
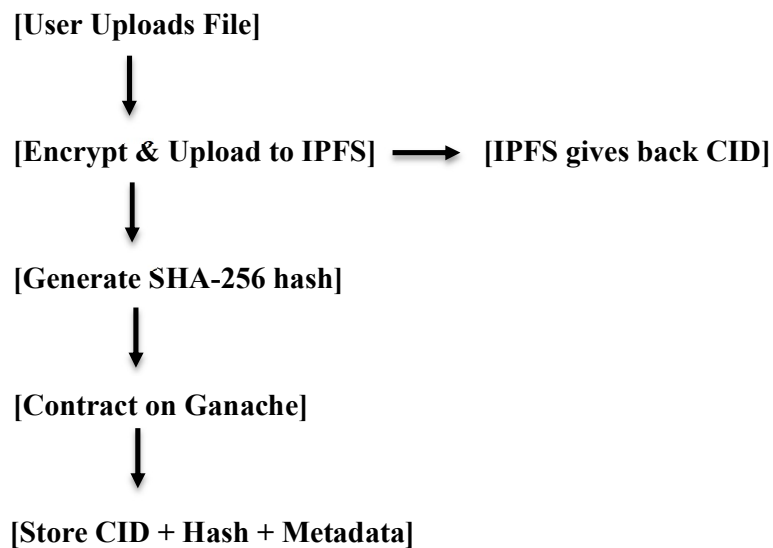


Fig1: Document Uploading Process



FlowChart3: Encryption Flow

Step-by-step:

1. User selects a legal document and provides:
 - Secret Key
 - Case ID
 - Title & Category
2. The file is **converted to base64** and **AES-encrypted** using the secret key.
3. A **SHA-256 hash** of the original file is generated to act as its fingerprint.
4. The **encrypted file is uploaded to IPFS**, which returns a **CID** (Content ID).
5. A **smart contract is triggered** on Ethereum with:
 - Case ID
 - IPFS CID
 - SHA-256 Hash
 - Title & Category
6. A **QR code** is generated pointing to:

4.2 Decrypting and Downloading Document

 **Secure Verdict Vault**

Case ID:

T12E436Y

Document Title:

Agreement

Category:

Debt

Choose File:

Choose File

example.txt

Enter Secret Key:

Encrypt & Upload

Enter Secret Key to Decrypt:

Download & Decrypt

Verify by Case ID:

T12E436Y

Verify

Found: IPFS Hash - QmVkt1pH3HgtcjPjc8JZRMxunKJAIQ3w4sZhtQ3aEFzn9, Uploader: 0x352Ac22e8f85bD1B4B0560b51670a440ECBb54F3

Fetching and decrypting...

TX Hash: 0xc0f75bc15f3ca3f5caa405d666b9f2f44809d44ccc1443e189638345b38616b0

Scan QR code to download file:



Fig2: Document Decrypting Process

ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS	EVENTS	LOGS	SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 14	GAS PRICE 20000000000	GAS LIMIT 6721975	HARDFORK MERGE	NETWORK ID 5777	RPC SERVER HTTP://127.0.0.1:7545	MINING STATUS AUTOMINING
						WORKSPACE VERDICTVAULT
						SWITCH ⚙
BLOCK 14	MINED ON 2025-04-17 10:47:52				GAS USED 163040	1 TRANSACTION
BLOCK 13	MINED ON 2025-04-17 10:43:06				GAS USED 163040	1 TRANSACTION
BLOCK 12	MINED ON 2025-04-17 10:39:14				GAS USED 163016	1 TRANSACTION
BLOCK 11	MINED ON 2025-04-17 10:38:40				GAS USED 26941	1 TRANSACTION
BLOCK 10	MINED ON 2025-04-16 10:14:53				GAS USED 163052	1 TRANSACTION
BLOCK 9	MINED ON 2025-04-16 10:14:20				GAS USED 26941	1 TRANSACTION
BLOCK 8	MINED ON 2025-04-16 10:10:22				GAS USED 163040	1 TRANSACTION
BLOCK 7	MINED ON 2025-04-09 14:27:23				GAS USED 163064	1 TRANSACTION
BLOCK 6	MINED ON 2025-04-09 14:26:55				GAS USED 26941	1 TRANSACTION

Fig3: Transactions Proof

[Case ID] $\longrightarrow$ Smart Contract $\longrightarrow$ [IPFS CID + SHA-256 + Metadata]



Fetch Encrypted File from IPFS



Verify SHA-256 Hash Matches



Decrypt File with Secret Key

FlowChart4: Decryption Flow

When someone wants to retrieve and decrypt:

1. They go to the /decrypt page (via QR or manually).
2. Enter the **Case ID**.
3. The smart contract returns the:
 - CID
 - SHA-256 hash
 - Title & Category
4. The file is fetched from IPFS using the **CID**.
5. User enters the **same secret key** to decrypt.
6. A new **SHA-256 hash is generated** from the decrypted file.
7. System **compares it** with the hash from the smart contract:
 - If it matches: File is genuine and untampered
 - If it doesn't: File or key is incorrect

5 CONCLUSION AND FUTURE WORK

5.1 Conclusion

Verdict Vault has been successfully developed as a secure and compliant digital evidence management system that ensures the integrity, confidentiality, and accessibility of digital evidence throughout its lifecycle. The system is built to meet the needs of law enforcement, legal professionals, and organizations requiring a reliable tool for managing evidence securely.

Key achievements include:

- **Tamper-proof storage:** Using encryption to protect evidence from unauthorized access.
- **Auditable workflows:** Ensuring that every action on evidence is logged for accountability.
- **Role-based access control:** Protecting sensitive evidence by limiting user access based on roles.

Verdict Vault represents a significant step forward in digital evidence management, ensuring compliance with legal standards while providing a robust solution for forensic investigations.

5.2 Future Work

To further improve Verdict Vault, the following enhancements are planned:

- **Mobile Application:** Develop a mobile app to allow investigators to access evidence on the go.
- **AI-Based Evidence Classification:** Use machine learning algorithms to classify and tag evidence for easier retrieval.
- **Blockchain Integration:** Implement blockchain to further enhance the immutability of evidence records.
- **Multilingual Support:** Add support for multiple languages to make the system accessible to international users.
- **Expanded File Formats:** Support a broader range of file types for evidence submission and retrieval.
- **Zero-Knowledge Proofs (ZKPs):** To prove ownership/access without revealing the document.
- **Decentralized Identity (DID):** For secure user identification.
- **Integration with Court Management Systems:** To streamline legal workflows.

6 REFERENCES

- "Digital Evidence and Computer Crime" by Eoghan Casey
- "Computer Forensics: Investigating Network Intrusions and Cybercrime" by Chris Prosise and Kevin Mandia
- ISO/IEC 27001:2013 (Information Security Management Systems)
- NIST Special Publication 800-53 (Security and Privacy Controls for Information Systems and Organizations)
- IPFS Documentation: <https://docs.ipfs.tech>
- Ethereum Smart Contracts (Solidity): <https://docs.soliditylang.org>
- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>

7. APPENDIX

7.1 Features Table

Feature	Description
Secure Upload	AES encryption of files using secret key
Tamper Detection	SHA-256 hash comparison
IPFS Integration	Decentralized, immutable file storage
Ethereum Smart Contract	On-chain hash + metadata logging
QR Code Access	Quick retrieval via unique QR
Role-Based Access	Investigator/Admin control levels
Full Audit Trail	All actions are time-stamped and logged

Table2: Features Table